

# Course Overview and The Shell

## CMSC398W: Practical Tools For Efficient Development

Mohammad Durrani

January 26, 2026

# About The Class

## Course Information

### CMSC398W: Practical Tools For Efficient Development

- Overview of tools used in development: command line, Git, debuggers, build systems, etc.
- Emphasis on breadth over depth
- Hands-on learning through projects

## Course Goals

- Improve your computing ecosystem literacy
- Improve your efficiency
- Reduce cognitive load while developing

# About Me - Mohammad

## Mohammad Durrani

- Senior
- CS + Math, minor in Robotics
- **Previous Experience:**
  - Teaching this class since Spring 2025
  - SWE Intern at Google in SF
  - SWE Intern at TRX Systems in Greenbelt

## Hobbies:

- Rock climbing
- Basketball
- Robots

# What To Expect

## Course Structure

Read the syllabus for all course related information!

- Introduction of topic
- Motivation / Real world example
- Technical details
- Projects focusing on application
- Best way to learn: **use them**

# Course Logistics

## Communication & Submission

- All course communication: **Piazza**
- All assignments submitted: **Gradescope**
- **10% per-day late penalty** (except the last project)

## Grade Breakdown

| Percentage | Title                     | Description                    |
|------------|---------------------------|--------------------------------|
| 80%        | Projects (20% per)        | 4 Major Projects               |
| 15%        | Application Days (5% per) | Completion of Application Days |
| 5%         | Participation             | Participation in class         |

# Application Days

## Based on Feedback

Based on feedback from the previous iteration, we'll have days focused on using the tools we learn in class on toy problems.

# Application Days

## Based on Feedback

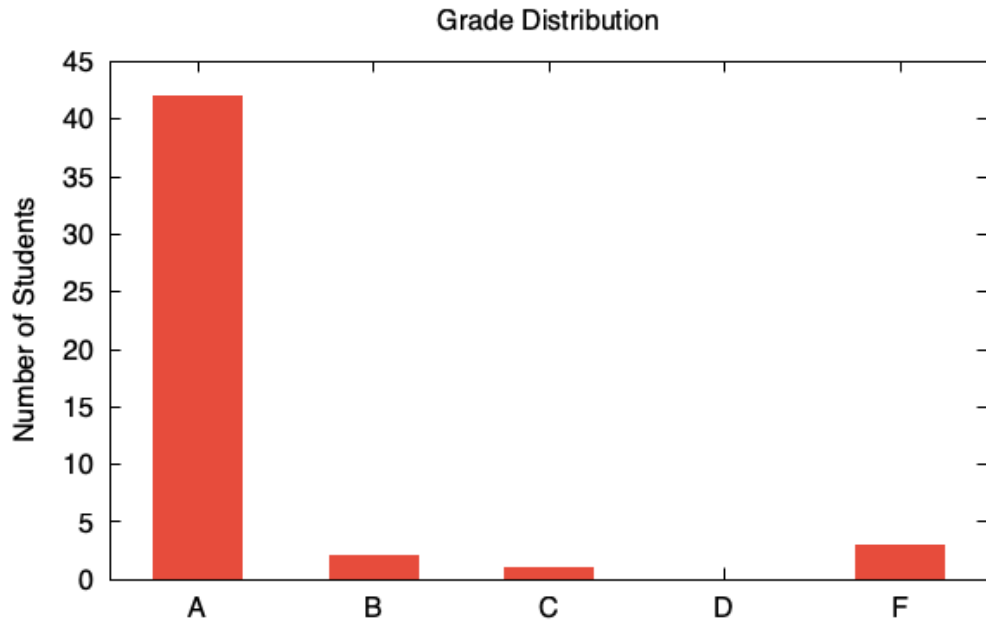
Based on feedback from the previous iteration, we'll have days focused on using the tools we learn in class on toy problems.

## Three Application Days

- 1 Shell
- 2 Git
- 3 Networking

(Subject to change)

# Grade Distribution





# Introduction

## Why The Shell?

- Traditional interfaces: graphical UIs, voice commands, AR/VR

# Introduction

## Why The Shell?

- Traditional interfaces: graphical UIs, voice commands, AR/VR
- Modern interfaces great for ~80% of use cases, but can be limiting

## Focus

- We'll focus on **bash** (Bourne Again Shell)
- One of the most widely used shells
- Essential for developers, system administrators, and power users

# Introduction

## Why The Shell?

- Traditional interfaces: graphical UIs, voice commands, AR/VR
- Modern interfaces great for ~80% of use cases, but can be limiting
- You cannot click a button that doesn't exist

## Focus

- We'll focus on **bash** (Bourne Again Shell)
- One of the most widely used shells
- Essential for developers, system administrators, and power users

# Introduction

## Why The Shell?

- Traditional interfaces: graphical UIs, voice commands, AR/VR
- Modern interfaces great for ~80% of use cases, but can be limiting
- You cannot click a button that doesn't exist
- You cannot give a voice command that hasn't been programmed

## Focus

- We'll focus on **bash** (Bourne Again Shell)
- One of the most widely used shells
- Essential for developers, system administrators, and power users

# Introduction

## Why The Shell?

- Traditional interfaces: graphical UIs, voice commands, AR/VR
- Modern interfaces great for ~80% of use cases, but can be limiting
- You cannot click a button that doesn't exist
- You cannot give a voice command that hasn't been programmed
- The shell provides a powerful **text-based interface** with flexibility and control

## Focus

- We'll focus on **bash** (Bourne Again Shell)
- One of the most widely used shells
- Essential for developers, system administrators, and power users

# What is the Shell?

## Definition (Definition)

**Shell:** A program that allows you to execute commands and interface with your operating system.

## Characteristics

- Textual interface for computer interaction
- Most platforms have a shell with many types (bash, zsh, dash, fish, etc.)
- Requires a **terminal** / **terminal emulator** to interact with the shell
- We will use **bash** in this class (generally default on Linux)
- macOS / Windows have different defaults

# What is the Shell?

## Definition (Definition)

**Shell:** A program that allows you to execute commands and interface with your operating system.

## Characteristics

- Textual interface for computer interaction
- Most platforms have a shell with many types (bash, zsh, dash, fish, etc.)
- Requires a **terminal** / **terminal emulator** to interact with the shell
- We will use **bash** in this class (generally default on Linux)
- macOS / Windows have different defaults

## Reference

The **Bash Reference Manual** contains everything you need to know about bash!

# Important Terminology

## 1. Traditional Terminal

- Historical computing device
- Physical keyboard and screen connected to shared computer
- Text-only interface (pre-mouse era)
- Used specific protocols to communicate
- Evolution: colors, different fonts, extended character sets



# Important Terminology

## 1. Traditional Terminal

- Historical computing device
- Physical keyboard and screen connected to shared computer
- Text-only interface (pre-mouse era)
- Used specific protocols to communicate
- Evolution: colors, different fonts, extended character sets

## 2. Modern Terminal Emulator

Software that emulates traditional terminals

Examples by platform:

- **Windows:** CMD.exe, PowerShell, PuTTY, Windows Terminal
- **macOS:** Terminal.app, iTerm2
- **Linux:** xTerm, Gnome Terminal, Konsole, Kitty, Ghostty, Alacritty

# Terminal Emulator Features

## Modern Features

- Supports tabs, split panes, and Unicode
- Quality-of-life features:
  - Search functionality
  - Copy/paste
  - Customizable appearance

# Terminal Emulator Features

## Modern Features

- Supports tabs, split panes, and Unicode
- Quality-of-life features:
  - Search functionality
  - Copy/paste
  - Customizable appearance

## Definition (3. Shell/Command-line Interface)

Program started when logging into terminal that interprets commands and provides interactive environment.

# Motivation

## Why Master the Command Line?

- **Essential** for software development

# Motivation

## Why Master the Command Line?

- **Essential** for software development
- Makes you **much more efficient**

# Motivation

## Why Master the Command Line?

- **Essential** for software development
- Makes you **much more efficient**
- Many development tools are command line based

# Motivation

## Why Master the Command Line?

- **Essential** for software development
- Makes you **much more efficient**
- Many development tools are command line based
- Access remote servers and cloud platforms

# Motivation

## Why Master the Command Line?

- **Essential** for software development
- Makes you **much more efficient**
- Many development tools are command line based
- Access remote servers and cloud platforms
- Automate repetitive tasks



# Motivation

## Why Master the Command Line?

- **Essential** for software development
- Makes you **much more efficient**
- Many development tools are command line based
- Access remote servers and cloud platforms
- Automate repetitive tasks
- Work across different operating systems

# The Prompt

## Anatomy of the Prompt

```
username@domain directory$
```

| Component | Description                |
|-----------|----------------------------|
| username  | Currently logged in user   |
| @         | Separator (convention)     |
| domain    | Machine name               |
| directory | Current working directory  |
| \$        | User-level shell indicator |
| #         | Root-level shell indicator |

# The Prompt

## Anatomy of the Prompt

```
username@domain directory$
```

| Component | Description                |
|-----------|----------------------------|
| username  | Currently logged in user   |
| @         | Separator (convention)     |
| domain    | Machine name               |
| directory | Current working directory  |
| \$        | User-level shell indicator |
| #         | Root-level shell indicator |

## Note

The prompt is customizable and can include additional information (git status, time, etc.)

# Basic Commands

## How Commands Work

Inside a prompt you type a command, which is then executed or interpreted.

The shell:

- 1 Parses by splitting on whitespace
- 2 Runs the program indicated by the first word
- 3 Gives each subsequent word as an argument to the program

## Important Notes

- Commands are **case-sensitive**
- Each command is a separate program (usually)
- Arguments modify command behavior
- Special characters need proper handling
- Commands can accept flags/options starting with -

# Basic Commands - Examples

## Command Examples

```
# Show current date and time  
date
```

```
# Echo text to screen  
echo hello
```

```
# Handle spaces in arguments  
echo "Hello World"  
echo Hello\ World
```

```
# Multiple arguments  
echo first second third
```

# PATH Variable

## Definition (What is PATH?)

PATH is an environment variable that lists which directories the shell should search for programs.

## Characteristics

- Colon-separated list of directories
- Shell searches directories in order
- First matching executable is used
- Can be modified to add new directories
- Built-in commands take precedence over PATH

# PATH Variable - Example

## Exploring PATH

# Display PATH

```
cmssc398w:~$ echo $PATH  
/usr/local/sbin:/usr/local/bin:/usr/sbin:  
/usr/bin:/sbin:/bin:/snap/bin
```

# Find location of command

```
cmssc398w:~$ which echo  
/usr/bin/echo
```

# Run directly without PATH

```
cmssc398w:~$ /bin/echo $PATH  
/usr/local/sbin:/usr/local/bin:/usr/sbin:  
/usr/bin:/sbin:/bin:/snap/bin
```

# Modifying PATH

## Adding to PATH

```
export PATH="/Directory1:$PATH"
```

This adds /Directory1 to the beginning of your PATH, giving it priority in command searches.



# Navigating - Paths

- Start with / (Unix) or drive letter (Windows)
- Full path from root
- Work from any location
- **Example:** /home/user/docs

- Relative to current directory
- More concise
- Context-dependent

## Special Directories

| Symbol | Meaning           |
|--------|-------------------|
| .      | Current directory |
| ..     | Parent directory  |
| ~      | Home directory    |

# Essential Navigation Commands

## Commands

| Command             | Description             |
|---------------------|-------------------------|
| <code>pwd</code>    | Print working directory |
| <code>cd</code>     | Change directory        |
| <code>ls</code>     | List directory contents |
| <code>ls -l</code>  | Long format listing     |
| <code>ls -a</code>  | Show hidden files       |
| <code>ls -lh</code> | Human-readable sizes    |

# Navigation Examples

## Using Navigation Commands

# Show current directory

```
pwd
```

# Go to home directory

```
cd ~
```

```
cd
```

# Go to parent directory

```
cd ..
```

# Go to specific directory

```
cd /usr/local/bin
```

# Go to previous directory

```
cd -
```

# File and Directory Manipulation

## Common Operations

| Command            | Description                          |
|--------------------|--------------------------------------|
| <code>mkdir</code> | Make directory                       |
| <code>touch</code> | Create empty file / update timestamp |
| <code>cp</code>    | Copy files/directories               |
| <code>mv</code>    | Move/rename files                    |
| <code>rm</code>    | Remove files                         |
| <code>rmdir</code> | Remove empty directory               |

# File and Directory Manipulation

## Common Operations

| Command            | Description                          |
|--------------------|--------------------------------------|
| <code>mkdir</code> | Make directory                       |
| <code>touch</code> | Create empty file / update timestamp |
| <code>cp</code>    | Copy files/directories               |
| <code>mv</code>    | Move/rename files                    |
| <code>rm</code>    | Remove files                         |
| <code>rmdir</code> | Remove empty directory               |

## Important Flags

- `-r`: Recursive (for directories)
- `-i`: Interactive (ask before overwrite)
- `-f`: Force (no prompts)
- `-v`: Verbose (show what's happening)

# File Manipulation Examples

## Examples

# Create directory

```
mkdir new_folder
```

```
mkdir -p parent/child/grandchild
```

# Copy files

```
cp file.txt backup.txt
```

```
cp -r folder1 folder2
```

# Move/rename

```
mv old_name.txt new_name.txt
```

```
mv file.txt /path/to/destination/
```

# Remove files

```
rm file.txt
```

```
rm -r folder
```

```
rm -rf folder # Force remove (careful!)
```

# Viewing File Contents

## Viewing Commands

| Command              | Description                   |
|----------------------|-------------------------------|
| <code>cat</code>     | Display entire file           |
| <code>less</code>    | Page through file (q to quit) |
| <code>head</code>    | Show first 10 lines           |
| <code>tail</code>    | Show last 10 lines            |
| <code>tail -f</code> | Follow file updates (logs)    |

# Viewing Examples

## File Viewing

# Display entire file

```
cat file.txt
```

# Page through file

```
less file.txt
```

# First 20 lines

```
head -n 20 file.txt
```

# Last 15 lines

```
tail -n 15 file.txt
```

# Follow log file

```
tail -f /var/log/system.log
```



# Environment Variables

## Definition (What are Environment Variables?)

Variables that affect how processes run and are accessible to all child processes.

## Common Variables

| Variable | Description           |
|----------|-----------------------|
| PATH     | Command search path   |
| HOME     | User's home directory |
| USER     | Current username      |
| SHELL    | Current shell         |
| PWD      | Current directory     |

# Environment Variable Examples

## Working with Variables

```
# Display variable
```

```
echo $PATH
```

```
echo $HOME
```

```
# Set variable (current session)
```

```
MY_VAR="value"
```

```
echo $MY_VAR
```

```
# Export variable (to child processes)
```

```
export MY_VAR="value"
```

```
# View all environment variables
```

```
env
```

```
printenv
```

# Searching and Finding

## Search Commands

| Command             | Description                      |
|---------------------|----------------------------------|
| <code>find</code>   | Find files by name/type/size     |
| <code>grep</code>   | Search file contents             |
| <code>which</code>  | Locate command                   |
| <code>locate</code> | Fast file search (uses database) |

# Search Examples

## Finding Files and Content

# Find files by name

```
find . -name "*.txt"
```

```
find /home -name "config.json"
```

# Find directories

```
find . -type d -name "test*"
```

# Search file contents

```
grep "error" logfile.txt
```

```
grep -r "TODO" . # Recursive search
```

```
grep -i "warning" file.txt # Case-insensitive
```

# Locate command

```
which python
```

# Advanced grep

## grep Options

| Option | Description                      |
|--------|----------------------------------|
| -i     | Case-insensitive search          |
| -r     | Recursive search                 |
| -v     | Invert match (show non-matching) |
| -n     | Show line numbers                |
| -c     | Count matching lines             |
| -l     | Show only filenames              |
| -A N   | Show N lines after match         |
| -B N   | Show N lines before match        |
| -C N   | Show N lines before and after    |

# grep Examples

## Advanced grep Usage

# Find all TODO comments with context

```
grep -n -C 2 "TODO" *.py
```

# Count errors in log

```
grep -c "ERROR" application.log
```

# Find files containing pattern

```
grep -rl "import numpy" .
```

# Complex pattern matching

```
grep -E "error|warning|critical" logfile.txt
```

# Exclude pattern

```
grep -v "DEBUG" logfile.txt | less
```

# Wildcards and Globbing

## Glob Patterns

| Pattern | Matches                       |
|---------|-------------------------------|
| *       | Any string (including empty)  |
| ?       | Any single character          |
| [abc]   | Any character in brackets     |
| [a-z]   | Any character in range        |
| [!abc]  | Any character NOT in brackets |

# Globber Examples

## Pattern Matching

# All .txt files

```
ls *.txt
```

# Files starting with 'test'

```
ls test*
```

# Single character wildcard

```
ls file?.txt
```

# Character sets

```
ls file[123].txt
```

```
ls [a-z]*.txt
```

# Negation

```
ls [!a-z]*
```



# Brace Expansion

## Definition (Creating Multiple Arguments)

Brace expansion generates multiple strings from a pattern.

## Examples

```
# Create multiple files
```

```
touch file{1,2,3}.txt
```

```
# Creates: file1.txt, file2.txt, file3.txt
```

```
# Ranges
```

```
echo {1..10}
```

```
echo {a..z}
```

```
# Nested expansion
```

```
mkdir -p project/{src,test,docs}/{main,utils}
```

```
# Combinations
```

```
echo {A,B}{1,2}
```

```
# Substitution: A1, A2, B1, B2
```

# Command History

## History Features

| Key/Command    | Action                                    |
|----------------|-------------------------------------------|
| Up/Down arrows | Navigate history                          |
| history        | Show command history                      |
| !n             | Execute command number n                  |
| !!             | Repeat last command                       |
| !string        | Execute last command starting with string |
| Ctrl+R         | Reverse search history                    |

# History Examples

## Using History

# View history

history

history 20 # Last 20 commands

# Execute from history

!42 # Run command number 42

!! # Repeat last command

!grep # Last command starting with grep

# Search history

Ctrl+R # Then start typing

# Clear history

history -c

# Tab Completion

## Productivity Booster

Tab completion saves time and prevents typos:

- Press Tab to complete commands, files, directories

# Tab Completion

## Productivity Booster

Tab completion saves time and prevents typos:

- Press Tab to complete commands, files, directories
- Press Tab twice to see all possibilities

## Example Workflow

Type `cd /usr/loc` then press Tab → completes to `cd /usr/local/`

# Tab Completion

## Productivity Booster

**Tab completion** saves time and prevents typos:

- Press Tab to complete commands, files, directories
- Press Tab twice to see all possibilities
- Works with commands, paths, and filenames

## Example Workflow

Type `cd /usr/loc` then press Tab → completes to `cd /usr/local/`

# Tab Completion

## Productivity Booster

**Tab completion** saves time and prevents typos:

- Press Tab to complete commands, files, directories
- Press Tab twice to see all possibilities
- Works with commands, paths, and filenames
- Context-aware in many shells

## Example Workflow

Type `cd /usr/loc` then press Tab → completes to `cd /usr/local/`

# Command Substitution

## Definition (Using Command Output)

Command substitution allows you to use the output of a command as an argument to another command.



# Command Substitution Examples

## Substitution Syntax

```
# Using $(command) syntax (preferred)
echo "Today is $(date)"
files=$(ls -l)
echo "Found $(wc -l < file.txt) lines"
```

```
# Using backticks (older syntax)
echo "Today is `date`"
```

```
# Nested substitution
echo "User $(whoami) in $(pwd)"
```

# Process Substitution

## Definition (Advanced Input/Output)

Process substitution allows you to use the output of a command as if it were a file.

## Examples

```
# Compare output of two commands
```

```
diff <(ls dir1) <(ls dir2)
```

```
# Use command output as input file
```

```
while read line; do
```

```
    echo "Processing: $line"
```

```
done < <(find . -name "*.txt")
```

```
# Multiple inputs
```

```
paste <(cut -f1 file1.csv) <(cut -f2 file2.csv)
```

# Redirection Basics

## Redirection Operators

| Operator | Description                     |
|----------|---------------------------------|
| >        | Redirect output (overwrite)     |
| »        | Redirect output (append)        |
| <        | Redirect input                  |
| 2>       | Redirect stderr                 |
| 2>&1     | Redirect stderr to stdout       |
| &>       | Redirect both stdout and stderr |

# Redirection Examples

## Using Redirection

```
# Redirect output to file
echo "Hello" > output.txt
ls -l >> listing.txt
```

```
# Redirect input
sort < unsorted.txt
```

```
# Redirect errors
command 2> errors.log
```

```
# Combine stdout and stderr
command > all.log 2>&1
command &> all.log # Shorter syntax
```

# Pipes

## Definition (What are Pipes?)

Pipes (|) connect the output of one command to the input of another, enabling powerful command chaining.

## Pipe Philosophy

Unix philosophy: "Do one thing and do it well"  
Combine simple tools to accomplish complex tasks!

# Pipe Examples

## Basic Piping

# Count lines in directory listing

```
ls -l | wc -l
```

# Search and sort

```
cat file.txt | grep "pattern" | sort
```

# Find largest files

```
du -h | sort -rh | head -10
```

# Process logs

```
cat access.log | grep "404" | wc -l
```

# Advanced Pipe Chains

## Complex Pipelines

```
# Top 10 most common IP addresses in log
```

```
cat access.log \  
  | awk '{print $1}' \  
  | sort \  
  | uniq -c \  
  | sort -rn \  
  | head -10
```

```
# Find largest directories
```

```
du -sh * \  
  | sort -rh \  
  | head -5
```

# tee Command

## Definition (Read and Write Simultaneously)

The tee command reads from stdin and writes to both stdout and files.

## Examples

```
# Save output to file while displaying
ls -la | tee directory_listing.txt
```

```
# Append to file
command | tee -a logfile.txt
```

```
# Write to multiple files
command | tee file1.txt file2.txt
```

```
# Use with sudo
echo "new line" | sudo tee -a /etc/hosts
```



# What is xargs?

## Definition (Definition)

`xargs` builds and executes command lines from standard input, converting input into arguments for another command.

## Why xargs?

Many commands don't read from stdin they need arguments:

- `rm`, `cp`, `mv` expect filenames as arguments
- `xargs` bridges this gap
- Enables parallel execution
- Handles large argument lists safely

# xargs Basic Usage

## Simple Examples

# Delete all .tmp files

```
find . -name "*.tmp" | xargs rm
```

# Move files to directory

```
ls *.txt | xargs -I {} mv {} archive/
```

# Print each item on new line

```
echo "one two three" | xargs -n 1
```

# Execute command per line

```
cat urls.txt | xargs -n 1 curl -O
```

# xargs Options

## Common Flags

| Option | Description                    |
|--------|--------------------------------|
| -n N   | Use at most N args per command |
| -I {}  | Replace {} with input          |
| -P N   | Run N processes in parallel    |
| -t     | Print command before executing |
| -p     | Prompt before executing        |
| -0     | Input items null-terminated    |
| -L N   | Use N lines per command        |

# xargs Advanced Examples

## Powerful Patterns

# Parallel processing (4 jobs at once)

```
find . -name "*.jpg" | xargs -P 4 -I {} convert {} {}.png
```

# Handle filenames with spaces

```
find . -name "*.txt" -print0 | xargs -0 grep "pattern"
```

# Backup files with custom names

```
ls *.conf | xargs -I {} cp {} {}.backup
```

# Download files in parallel

```
cat urls.txt | xargs -n 1 -P 8 wget
```

# Convert multiple files

```
ls *.md | xargs -I {} pandoc {} -o {}.pdf
```

## xargs with find

### The Power Combo

# Delete files older than 30 days

```
find /tmp -mtime +30 -print0 | xargs -0 rm
```

# Change permissions on all scripts

```
find . -name "*.sh" -print0 | xargs -0 chmod +x
```

# Search in found files

```
find . -name "*.log" | xargs grep "ERROR"
```

# Archive old files

```
find . -mtime +365 -print0 | \  
xargs -0 tar -czf archive.tar.gz
```

## xargs vs exec

### Comparison

# Using find -exec (one process per file)

```
find . -name "*.txt" -exec grep "pattern" {} \;
```

# Using xargs (batches arguments, more efficient)

```
find . -name "*.txt" | xargs grep "pattern"
```

# With null termination for safety

```
find . -name "*.txt" -print0 | xargs -0 grep "pattern"
```

## xargs vs exec

### Comparison

# Using find -exec (one process per file)

```
find . -name "*.txt" -exec grep "pattern" {} \;
```

# Using xargs (batches arguments, more efficient)

```
find . -name "*.txt" | xargs grep "pattern"
```

# With null termination for safety

```
find . -name "*.txt" -print0 | xargs -0 grep "pattern"
```

### When to Use Each

- Use xargs for better performance with many files
- Use find -exec when you need per-file control

# Introduction to Data Wrangling

## Definition (What is Data Wrangling?)

Data wrangling is the process of cleaning, transforming, and analyzing data using command-line tools.

## Common Tasks

- Extracting specific fields from logs
- Counting occurrences and frequencies
- Filtering and transforming data
- Aggregating statistics
- Reformatting output



# Text Processing Tools

## Essential Tools

| Command           | Description                     |
|-------------------|---------------------------------|
| <code>cat</code>  | Concatenate files               |
| <code>sort</code> | Sort lines                      |
| <code>uniq</code> | Remove duplicate lines          |
| <code>wc</code>   | Word/line/character count       |
| <code>cut</code>  | Extract columns                 |
| <code>tr</code>   | Translate or delete characters  |
| <code>sed</code>  | Stream editor                   |
| <code>awk</code>  | Pattern scanning and processing |

# Text Processing Examples

## Basic Operations

# Count lines in file

```
wc -l file.txt
```

# Sort and remove duplicates

```
sort file.txt | uniq
```

# Extract specific column

```
cut -d',' -f2 data.csv
```

# Count unique values

```
sort file.txt | uniq -c
```

# Translate characters

```
echo "hello" | tr 'a-z' 'A-Z'
```

# sort Command

## Sorting Options

| Option | Description                     |
|--------|---------------------------------|
| -r     | Reverse order                   |
| -n     | Numeric sort                    |
| -h     | Human-readable numbers (1K, 2M) |
| -k N   | Sort by Nth field               |
| -t     | Field delimiter                 |
| -u     | Unique (remove duplicates)      |

# sort Examples

## Sorting Data

# Sort numerically in reverse

```
sort -rn numbers.txt
```

# Sort by second column

```
sort -k2 data.txt
```

# Sort CSV by third column

```
sort -t',' -k3 data.csv
```

# Sort file sizes

```
du -sh * | sort -rh
```

## uniq Command

## Unique Line Options

| Option | Description             |
|--------|-------------------------|
| -c     | Prefix lines with count |

# uniq Examples

## Finding Duplicates

# Count occurrences

```
sort file.txt | uniq -c
```

# Find duplicates

```
sort file.txt | uniq -d
```

# Case-insensitive unique

```
sort -f file.txt | uniq -i
```

# Most common lines

```
sort file.txt | uniq -c | sort -rn
```

# cut and paste

## Column Manipulation

# Extract columns 1 and 3

```
cut -d',' -f1,3 data.csv
```

# Extract character range

```
cut -c1-10 file.txt
```

# Combine columns from different files

```
paste file1.txt file2.txt
```

# Use custom delimiter

```
paste -d',' file1.txt file2.txt
```

# tr Command

## Character Translation

# Convert to uppercase

```
echo "hello" | tr 'a-z' 'A-Z'
```

# Delete characters

```
echo "hello123" | tr -d '0-9'
```

# Squeeze repeated characters

```
echo "hello    world" | tr -s ' '
```

# Replace spaces with newlines

```
echo "one two three" | tr ' ' '\n'
```

# ROT13 encoding

```
echo "hello" | tr 'a-zA-Z' 'n-zA-mN-ZA-M'
```

# Data Wrangling Example

## Real-World Pipeline

```
# Analyze web server logs
# Find most common ERROR messages
cat access.log \
  | grep "ERROR" \
  | cut -d' ' -f5- \
  | sort \
  | uniq -c \
  | sort -rn \
  | head -10
```

This pipeline:

- 1 Reads the log file
- 2 Filters ERROR lines
- 3 Extracts relevant fields
- 4 Sorts entries
- 5 Counts unique occurrences



# Regular Expressions Basics

## Definition (What are Regular Expressions?)

Regular expressions (regex) are patterns used to match character combinations in strings.

## Common Patterns

| Pattern | Matches                       |
|---------|-------------------------------|
| .       | Any single character          |
| *       | Zero or more of previous      |
| +       | One or more of previous       |
| ?       | Zero or one of previous       |
| ^       | Start of line                 |
| \$      | End of line                   |
| [abc]   | Any character in brackets     |
| [^abc]  | Any character NOT in brackets |

# Advanced Regex Patterns

## More Patterns

| Pattern                          | Matches                               |
|----------------------------------|---------------------------------------|
| <code>\d</code>                  | Any digit (0-9)                       |
| <code>\w</code>                  | Any word character (a-z, A-Z, 0-9, _) |
| <code>\s</code>                  | Any whitespace                        |
| <code>(...)</code>               | Capture group                         |
| <code>...{{{vert()}}}\...</code> | Alternative (or)                      |
| <code>{n}</code>                 | Exactly n times                       |
| <code>{n,m}</code>               | Between n and m times                 |

# Introduction to sed

## Definition (What is sed?)

sed (Stream Editor) is a powerful tool for parsing and transforming text in a data stream or file.

## Common Uses

- Text substitution
- Line deletion
- In-place file editing
- Pattern-based text manipulation
- Automated text transformations

## sed Basic Syntax

## sed Command Structure

```
sed 'command' file
```

```
sed -e 'command1' -e 'command2' file
```

```
sed -i 'command' file # In-place editing
```

# sed Substitution

## Basic Substitution

# Basic substitution (first occurrence)

```
sed 's/old/new/' file.txt
```

# Global substitution (all occurrences)

```
sed 's/old/new/g' file.txt
```

# Case-insensitive

```
sed 's/old/new/gi' file.txt
```

# Only on specific lines

```
sed '3s/old/new/' file.txt
```

```
sed '1,5s/old/new/g' file.txt
```

# With different delimiter

```
sed 's|/path/old|/path/new|g' file.txt
```

# sed Advanced Substitution

## Using Capture Groups

# Capture and reuse with \1, \2, etc.

```
echo "Hello World" | sed 's/\(.*\) \(.*\)/\2 \1/'
```

# Output: World Hello

# Swap two columns

```
sed 's/\([^,]*\),\([^,]*\)/\2,\1/' data.csv
```

# Add parentheses around numbers

```
echo "Error 404" | sed 's/\([0-9]\+\)/(\1)/'
```

# Output: Error (404)

# Extract domain from URL

```
echo "https://example.com/path" | \
sed 's|https://\([^/]*\)/*.*|\1|'
```

# sed Line Operations

### Deleting and Printing Lines :BEAMER<sub>ENV</sub>: block

# Delete lines

```
sed '3d' file.txt           # Delete line 3
sed '1,5d' file.txt         # Delete lines 1-5
sed '/pattern/d' file.txt   # Delete matching lines
sed '/^$/d' file.txt        # Delete empty lines
```

# Print lines

```
sed -n '3p' file.txt        # Print only line 3
sed -n '1,5p' file.txt      # Print lines 1-5
sed -n '/pattern/p' file.txt # Print matching lines
```

# Print non-matching lines

```
sed -n '/pattern/!p' file.txt
```

# sed Insert and Append

## Adding Text

# Insert before line

```
sed '3i\New text' file.txt
```

# Append after line

```
sed '3a\New text' file.txt
```

# Insert before pattern

```
sed '/pattern/i\New text' file.txt
```

# Append after pattern

```
sed '/pattern/a\New text' file.txt
```

# Change entire line

```
sed '3c\Replacement line' file.txt
```

```
sed '/pattern/c\Replacement' file.txt
```

# sed In-Place Editing

## Modifying Files Directly

# Edit file in place

```
sed -i 's/old/new/g' file.txt
```

# Create backup before editing

```
sed -i.bak 's/old/new/g' file.txt
```

# Multiple operations

```
sed -i -e 's/foo/bar/g' -e 's/hello/world/g' file.txt
```

# Process multiple files

```
sed -i 's/old/new/g' *.txt
```



# sed Address Ranges

## Working with Line Ranges

```
# Specific line range
sed '10,20s/old/new/g' file.txt

# From line to end
sed '10,$s/old/new/g' file.txt

# Pattern to pattern
sed '/START/,/END/s/old/new/g' file.txt

# Every Nth line
sed '0~3s/old/new/g' file.txt # Every 3rd line

## sed Practical Examples :BEAMEROPT: fragile
```

## Real-World Usage

```
# Remove comments
sed 's/#.*//' config.txt
```

# Introduction to awk

## Definition (What is awk?)

awk is a powerful pattern-scanning and processing language, excellent for structured text and data extraction.

## When to Use awk

- Processing column-based data (CSV, TSV, logs)
- Performing calculations on data
- Generating reports
- Complex pattern matching
- Field-based transformations

# awk Basic Structure

## awk Syntax

```
awk 'pattern { action }' file  
awk '/pattern/ { print $0 }' file
```

## Built-in Variables

| Variable   | Description                           |
|------------|---------------------------------------|
| \$0        | Entire line                           |
| \$1, \$2.. | First field, second field, etc.       |
| NF         | Number of fields                      |
| NR         | Current record (line) number          |
| FS         | Field separator (default: whitespace) |
| RS         | Record separator (default: newline)   |

# awk Basic Examples

## Simple Usage

# Print entire file

```
awk '{ print }' file.txt
```

```
awk '{ print $0 }' file.txt
```

# Print specific column

```
awk '{ print $1 }' file.txt
```

# Print multiple columns

```
awk '{ print $1, $3 }' file.txt
```

# Print with custom separator

```
awk '{ print $1 "," $2 }' file.txt
```

# Print last column

```
awk '{ print $NF }' file.txt
```

# awk Field Separators

## Custom Delimiters

# Use comma as separator

```
awk -F',' '{ print $1 }' data.csv
```

# Use colon as separator

```
awk -F':' '{ print $1, $3 }' /etc/passwd
```

# Multiple character separator

```
awk -F':' '{ print $1 }' file.txt
```

# Tab separator

```
awk -F'\t' '{ print $1 }' file.tsv
```

# Set in BEGIN block

```
awk 'BEGIN { FS="," } { print $1 }' data.csv
```

# awk Pattern Matching

## Filtering with Patterns

# Match pattern in any field

```
awk '/error/' log.txt
```

# Match pattern in specific field

```
awk '$3 == "error"' log.txt
```

# Regular expression match

```
awk '$2 ~ /^[0-9]+$/' file.txt
```

# Negation

```
awk '$2 !~ /pattern/' file.txt
```

# Multiple conditions

```
awk '$1 == "ERROR" && $3 > 100' log.txt
```

```
awk '$1 == "ERROR" || $1 == "WARNING"' log.txt
```

# awk Numerical Operations

## Calculations

# Numeric comparison

```
awk '$3 > 100' file.txt
```

```
awk '$2 >= 50 && $2 <= 100' file.txt
```

# Arithmetic

```
awk '{ print $1, $2 * 1.1 }' file.txt
```

```
awk '{ print $1, $2 + $3 }' file.txt
```

# Sum a column

```
awk '{ sum += $2 } END { print sum }' file.txt
```

# Average

```
awk '{ sum += $2; count++ } END { print sum/count }' file.txt
```

# Count lines matching condition

```
awk '$3 > 100 { count++ } END { print count }' file.txt
```

# awk BEGIN and END

## Special Patterns

# BEGIN: executed before processing

```
awk 'BEGIN { print "Starting..." } { print $0 }' file.txt
```

# END: executed after processing

```
awk '{ sum += $1 } END { print "Total:", sum }' file.txt
```

# Both together

```
awk 'BEGIN { FS=","; print "Name,Score" }  
     { print $1, $2 }  
     END { print "Done" }' file.csv
```

# Initialize variables

```
awk 'BEGIN { max = 0 }  
     $2 > max { max = $2 }  
     END { print "Max:", max }' file.txt
```



# awk Built-in Functions

## Useful Functions

### # String functions

```
awk '{ print length($0) }' file.txt
awk '{ print toupper($1) }' file.txt
awk '{ print tolower($1) }' file.txt
awk '{ print substr($1, 1, 3) }' file.txt
```

### # Math functions

```
awk '{ print sqrt($1) }' file.txt
awk '{ print int($1) }' file.txt
```

### # Pattern matching

```
awk '{ if (match($0, /[0-9]+/)) print substr($0, RSTART, RLENGTH) }' file.txt
```

### # Substitution

```
awk '{ gsub(/old/, "new"); print }' file.txt
```

# awk Control Structures

## Conditionals and Loops

# If-else

```
awk '{ if ($3 > 100) print $1, "high"; else print $1, "low" }' file.txt
```

# For loop

```
awk '{ for (i=1; i<=NF; i++) print $i }' file.txt
```

# While loop

```
awk '{ i=1; while (i<=3) { print $i; i++ } }' file.txt
```

# Ternary operator

```
awk '{ print ($3 > 100) ? "high" : "low" }' file.txt
```

# awk Associative Arrays

## Hash Tables

# Count occurrences

```
awk '{ count[$1]++ } END { for (word in count) print word, count[word] }' file
```

# Sum by category

```
awk '{ sum[$1] += $2 } END { for (cat in sum) print cat, sum[cat] }' file.txt
```

# Find unique values

```
awk '!seen[$1]++' file.txt
```

# Build histogram

```
awk '{ hist[$1]++ } END { for (h in hist) print h, hist[h] }' log.txt
```

# awk Practical Examples

## Real-World Usage

# Calculate total and average

```
awk '{ sum += $2; count++ }  
    END { print "Total:", sum, "Avg:", sum/count }' data.txt
```

# Print lines with more than 5 fields

```
awk 'NF > 5' file.txt
```

# Print line numbers with content

```
awk '{ print NR ":", $0 }' file.txt
```

# Format output as table

```
awk '{ printf "%-10s %-10s %5d\n", $1, $2, $3 }' file.txt
```

# Extract email addresses

```
awk '/[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/' file.txt
```

# awk vs sed

### When to Use Which? :BEAMER\_ENV: block

- Simple substitutions
- Line-based operations
- In-place file editing
- Stream transformations

- Column-based data
- Calculations and math
- Complex pattern matching
- Report generation

## Definition (What is a Shell Script?)

A file containing a sequence of commands that the shell can execute.

## Key Features

- Automate repetitive tasks
- Combine multiple commands
- Add logic with conditionals and loops
- Use variables and functions
- Make tasks reproducible

# Creating a Script

## Basic Script Structure

```
#!/bin/bash  
# This is a comment  
  
echo "Hello, World!"  
echo "Current directory: $(pwd)"  
echo "User: $USER"
```

# Creating a Script

## Basic Script Structure

```
#!/bin/bash  
# This is a comment  
  
echo "Hello, World!"  
echo "Current directory: $(pwd)"  
echo "User: $USER"
```

## Making it Executable

```
chmod +x script.sh  
./script.sh
```



# Script Variables

## Using Variables

```
#!/bin/bash
```

```
# Define variables
```

```
name="Alice"
```

```
count=5
```

```
# Use variables
```

```
echo "Hello, $name"
```

```
echo "Count: $count"
```

```
# Command substitution
```

```
current_date=$(date)
```

```
echo "Today is: $current_date"
```

```
# Script arguments
```

```
echo "Script name: $0"
```

```
echo "First argument: $1"
```

# Special Variables

## Built-in Variables

| Variable | Description                           |
|----------|---------------------------------------|
| \$0      | Script name                           |
| \$1-\$9  | Positional arguments                  |
| @        | All arguments as separate words       |
| *        | All arguments as single word          |
| #        | Number of arguments                   |
| ?        | Exit status of last command           |
| \$       | Process ID of current shell           |
| !        | Process ID of last background command |

# Conditionals

## If Statements

```
#!/bin/bash
```

```
if [ -f "file.txt" ]; then
    echo "File exists"
elif [ -d "directory" ]; then
    echo "Directory exists"
else
    echo "Neither exists"
fi
```

```
# Numeric comparison
if [ $count -gt 10 ]; then
    echo "Greater than 10"
fi
```

```
# String comparison
if [ "$name" = "Alice" ]; then
```

# Test Operators

## File Tests

| Test                 | True if...                      |
|----------------------|---------------------------------|
| <code>-e file</code> | File exists                     |
| <code>-f file</code> | File exists and is regular file |
| <code>-d file</code> | File exists and is directory    |
| <code>-r file</code> | File exists and is readable     |
| <code>-w file</code> | File exists and is writable     |
| <code>-x file</code> | File exists and is executable   |
| <code>-s file</code> | File exists and is not empty    |

## ## Comparison Operators

## Numeric Comparisons

| Operator         | Description              |
|------------------|--------------------------|
| <code>-eq</code> | Equal to                 |
| <code>-ne</code> | Not equal to             |
| <code>-gt</code> | Greater than             |
| <code>-ge</code> | Greater than or equal to |

# Test Operators

## File Tests

| Test                 | True if...                      |
|----------------------|---------------------------------|
| <code>-e file</code> | File exists                     |
| <code>-f file</code> | File exists and is regular file |
| <code>-d file</code> | File exists and is directory    |
| <code>-r file</code> | File exists and is readable     |
| <code>-w file</code> | File exists and is writable     |
| <code>-x file</code> | File exists and is executable   |
| <code>-s file</code> | File exists and is not empty    |

## ## Comparison Operators

## Numeric Comparisons

| Operator         | Description              |
|------------------|--------------------------|
| <code>-eq</code> | Equal to                 |
| <code>-ne</code> | Not equal to             |
| <code>-gt</code> | Greater than             |
| <code>-ge</code> | Greater than or equal to |

# Loops

## For and While Loops

```
#!/bin/bash
```

```
# For loop
```

```
for i in {1..5}; do  
    echo "Number: $i"  
done
```

```
# Loop over files
```

```
for file in *.txt; do  
    echo "Processing $file"  
done
```

```
# C-style for loop
```

```
for ((i=0; i<5; i++)); do  
    echo "Count: $i"  
done
```

# Advanced Loop Techniques

### Loop Control :BEAMER<sub>ENV</sub>: block

# Break out of loop

```
for i in {1..10}; do
    if [ $i -eq 5 ]; then
        break
    fi
    echo $i
done
```

# Continue to next iteration

```
for i in {1..10}; do
    if [ $i -eq 5 ]; then
        continue
    fi
    echo $i
done
```

# Read file line by line

# Functions

## Defining Functions

```
#!/bin/bash
```

```
# Define function
```

```
greet() {  
    echo "Hello, $1!"  
}
```

```
# Call function
```

```
greet "Alice"  
greet "Bob"
```

```
# Function with return value
```

```
add() {  
    local result=$(( $1 + $2 ))  
    echo $result  
}
```



# Function Best Practices

## Local Variables and Returns

# Use local variables

```
calculate() {  
    local a=$1  
    local b=$2  
    local result=$((a + b))  
    echo $result  
}
```

# Return exit status

```
is_valid() {  
    if [ -f "$1" ]; then  
        return 0 # Success  
    else  
        return 1 # Failure  
    fi  
}
```

## Error Handling

```
### Robust Scripts :BEAMERENV: block
```

```
#!/bin/bash
```

```
# Exit on error
```

```
set -e
```

```
# Exit on undefined variable
```

```
set -u
```

```
# Pipeline fails if any command fails
```

```
set -o pipefail
```

```
# All together
```

```
set -euo pipefail
```

```
# Check command success
```

```
if ! command -v git &> /dev/null; then
```

```
    echo "git not found"
```

# Input and Output

## User Input

```
# Read user input
echo "Enter your name:"
read name
echo "Hello, $name"

# Read with prompt
read -p "Enter your age: " age

# Read password (hidden)
read -sp "Enter password: " password
echo

# Read with timeout
read -t 5 -p "Enter (5 sec timeout): " input

# Read into array
read -a array
```

# Arrays

## Working with Arrays

```
# Create array
fruits=("apple" "banana" "cherry")

# Access elements
echo ${fruits[0]}           # First element
echo ${fruits[@]}          # All elements
echo ${#fruits[@]}         # Array length

# Add elements
fruits+=("date")

# Iterate over array
for fruit in "${fruits[@]"; do
    echo $fruit
done

# Slice array
```

# String Manipulation

```
### String Operations :BEAMER_ENV: block
```

```
# Length
```

```
str="Hello World"
```

```
echo ${#str}
```

```
# Substring
```

```
echo ${str:0:5}          # Hello
```

```
echo ${str:6}           # World
```

```
# Replace
```

```
echo ${str/World/Bash}   # Replace first
```

```
echo ${str//l/L}         # Replace all
```

```
# Remove prefix/suffix
```

```
file="document.txt"
```

```
echo ${file%.txt}        # document
```

```
echo ${file#*.}          # txt
```

```
## Script Best Practices
```

# String Manipulation

```
### String Operations :BEAMER_ENV: block
```

```
# Length
```

```
str="Hello World"
```

```
echo ${#str}
```

```
# Substring
```

```
echo ${str:0:5}          # Hello
```

```
echo ${str:6}            # World
```

```
# Replace
```

```
echo ${str/World/Bash}   # Replace first
```

```
echo ${str//l/L}         # Replace all
```

```
# Remove prefix/suffix
```

```
file="document.txt"
```

```
echo ${file%.txt}        # document
```

```
echo ${file#*.}          # txt
```

```
## Script Best Practices
```

# String Manipulation

```
### String Operations :BEAMER_ENV: block
```

```
# Length
```

```
str="Hello World"
```

```
echo ${#str}
```

```
# Substring
```

```
echo ${str:0:5}          # Hello
```

```
echo ${str:6}           # World
```

```
# Replace
```

```
echo ${str/World/Bash}   # Replace first
```

```
echo ${str//l/L}         # Replace all
```

```
# Remove prefix/suffix
```

```
file="document.txt"
```

```
echo ${file%.txt}        # document
```

```
echo ${file#*.}          # txt
```

```
## Script Best Practices
```

# String Manipulation

```
### String Operations :BEAMER_ENV: block
```

```
# Length
```

```
str="Hello World"
```

```
echo ${#str}
```

```
# Substring
```

```
echo ${str:0:5}          # Hello
```

```
echo ${str:6}           # World
```

```
# Replace
```

```
echo ${str/World/Bash}   # Replace first
```

```
echo ${str//l/L}         # Replace all
```

```
# Remove prefix/suffix
```

```
file="document.txt"
```

```
echo ${file%.txt}        # document
```

```
echo ${file#*.}          # txt
```

```
## Script Best Practices
```



# String Manipulation

```
### String Operations :BEAMER_ENV: block
```

```
# Length
```

```
str="Hello World"
```

```
echo ${#str}
```

```
# Substring
```

```
echo ${str:0:5}          # Hello
```

```
echo ${str:6}           # World
```

```
# Replace
```

```
echo ${str/World/Bash}  # Replace first
```

```
echo ${str//l/L}        # Replace all
```

```
# Remove prefix/suffix
```

```
file="document.txt"
```

```
echo ${file%.txt}       # document
```

```
echo ${file#*.}         # txt
```

```
## Script Best Practices
```

# String Manipulation

```
### String Operations :BEAMER_ENV: block
```

```
# Length
```

```
str="Hello World"
```

```
echo ${#str}
```

```
# Substring
```

```
echo ${str:0:5}          # Hello
```

```
echo ${str:6}           # World
```

```
# Replace
```

```
echo ${str/World/Bash}   # Replace first
```

```
echo ${str//l/L}         # Replace all
```

```
# Remove prefix/suffix
```

```
file="document.txt"
```

```
echo ${file%.txt}        # document
```

```
echo ${file#*.}          # txt
```

```
## Script Best Practices
```

# String Manipulation

```
### String Operations :BEAMER_ENV: block
```

```
# Length
```

```
str="Hello World"
```

```
echo ${#str}
```

```
# Substring
```

```
echo ${str:0:5}          # Hello
```

```
echo ${str:6}           # World
```

```
# Replace
```

```
echo ${str/World/Bash}  # Replace first
```

```
echo ${str//l/L}        # Replace all
```

```
# Remove prefix/suffix
```

```
file="document.txt"
```

```
echo ${file%.txt}       # document
```

```
echo ${file#*.}         # txt
```

```
## Script Best Practices
```

# String Manipulation

```
### String Operations :BEAMER_ENV: block
```

```
# Length
```

```
str="Hello World"
```

```
echo ${#str}
```

```
# Substring
```

```
echo ${str:0:5}          # Hello
```

```
echo ${str:6}           # World
```

```
# Replace
```

```
echo ${str/World/Bash}   # Replace first
```

```
echo ${str//l/L}         # Replace all
```

```
# Remove prefix/suffix
```

```
file="document.txt"
```

```
echo ${file%.txt}        # document
```

```
echo ${file#*.}          # txt
```

```
## Script Best Practices
```

# String Manipulation

```
### String Operations :BEAMER_ENV: block
```

```
# Length
```

```
str="Hello World"
```

```
echo ${#str}
```

```
# Substring
```

```
echo ${str:0:5}          # Hello
```

```
echo ${str:6}           # World
```

```
# Replace
```

```
echo ${str/World/Bash}  # Replace first
```

```
echo ${str//l/L}        # Replace all
```

```
# Remove prefix/suffix
```

```
file="document.txt"
```

```
echo ${file%.txt}       # document
```

```
echo ${file#*.}         # txt
```

```
## Script Best Practices
```

# Debugging Scripts

## Debugging Techniques

# Print commands as they execute

```
set -x
```

# Turn off debug mode

```
set +x
```

# Run with debugging

```
bash -x script.sh
```

# Add debug statements

```
echo "DEBUG: Variable value is $var" >&2
```

# Check syntax without running

```
bash -n script.sh
```

# Verbose mode

```
set -v
```

# Practical Script Example

## Complete Script

```
#!/bin/bash
set -euo pipefail

# Backup script
backup_dir="/backups/$(date +%Y%m%d)"
source_dir="$HOME/documents"

# Create backup directory
mkdir -p "$backup_dir"

# Backup files
echo "Starting backup..."
tar -czf "$backup_dir/documents.tar.gz" "$source_dir"

if [ $? -eq 0 ]; then
    echo "Backup completed successfully"
else
```

# Questions?

{Thank you!}

Ask your questions on **Piazza**

Check the syllabus for assignment details